IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

¿Qué hay y dónde? Detección y segmentación

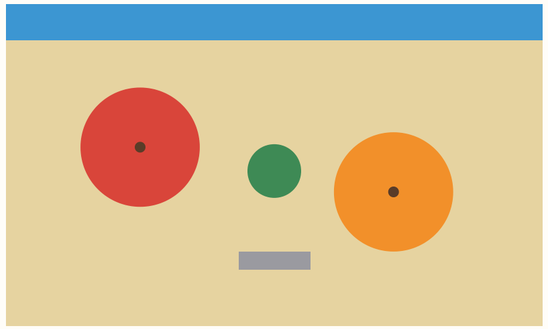
Teoría, práctica y, al final, material de consulta

Sprint 11 · S11_03 · teoría breve (15 min el 20/11), 35 min de práctica en clase y material de consulta

LA IDEA · TRES TAREAS

Tres tareas con imágenes

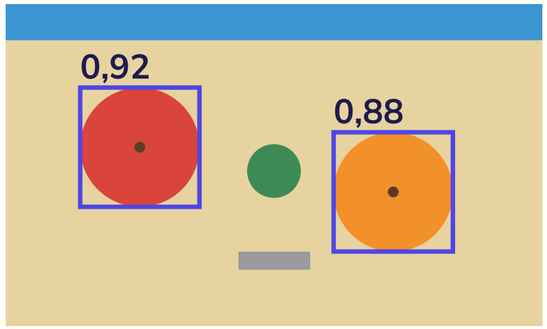
Clasificación · ¿qué hay?



Salida: una etiqueta

«playa»

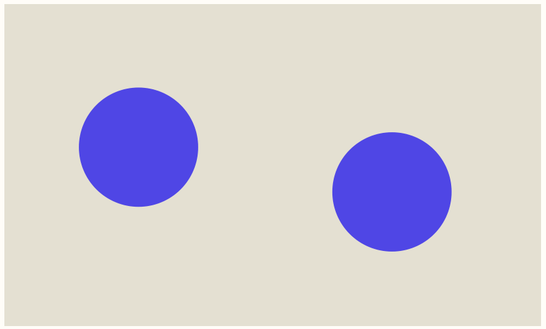
Detección · ¿qué y dónde?



Salida: cajas + clase + confianza

2 × «sombrilla»

Segmentación · ¿qué píxeles?



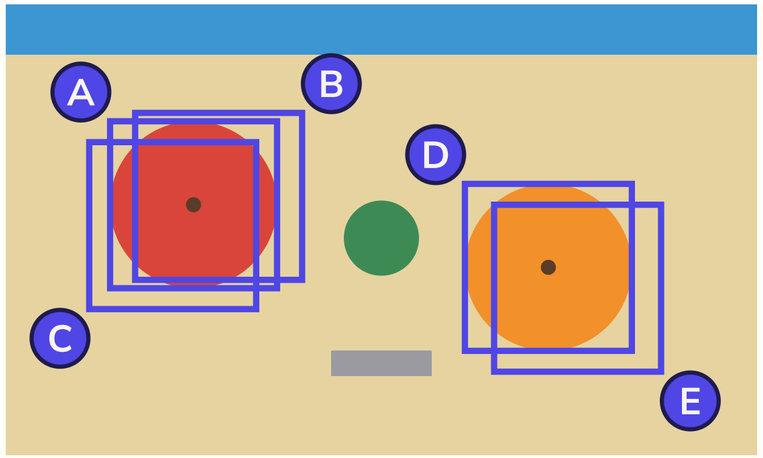
Salida: una máscara

píxeles de sombrilla en azul

TurisData: **contar sombrillas** es detección; **medir la superficie** de la piscina es segmentación.

Cómo trabaja un detector

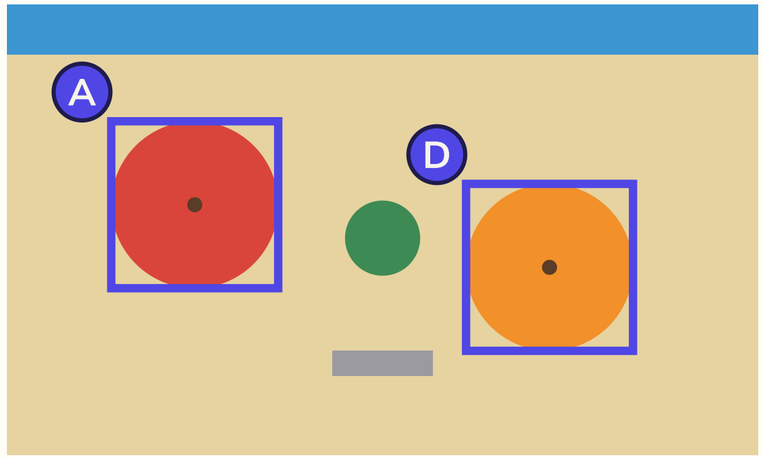
① Proponer: 5 cajas candidatas



cada una con clase y confianza

NMS →

② Limpiar: 2 cajas, una por objeto



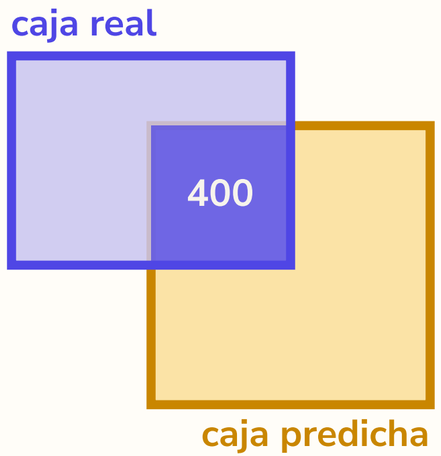
A y D se quedan

③ **Evaluar:** una caja acierta si su **IoU ≥ 0,5** con una caja real; de ahí salen precisión, recall y AP.

LA IDEA · LA IOU

IoU: cuánto se solapan dos cajas

Intersección ÷ unión



Cajas como [x1, y1, x2, y2]

Paso a paso

Áreas: **40 × 30 = 1 200** y **40 × 40 = 1 600**

Intersección: **20 × 20 = 400**

Unión: 1 200 + 1 600 − 400 = **2 400**

IoU = 400 / 2 400 ≈ 0,167

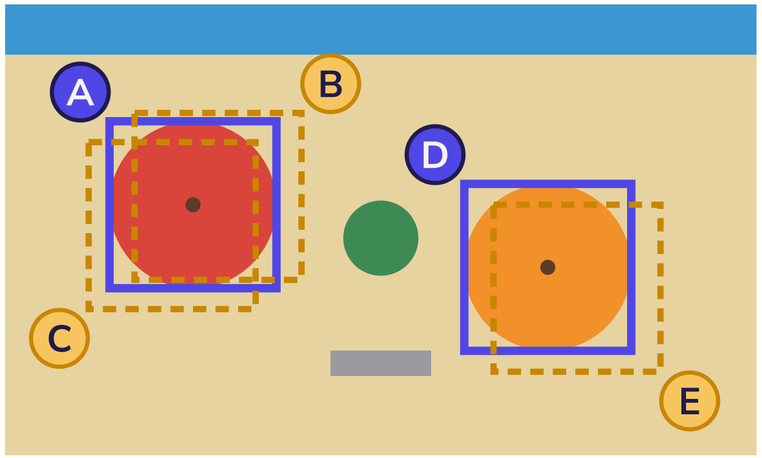
Caja predicha	IoU	¿Acierto (≥ 0,5)?
Casi encima	0,724	sí
La del dibujo	0,167	no
Lejos	0	no

Si las cajas no se tocan, el ancho de la intersección sale negativo (60 − 100 = −40): se deja en **0**.

LA IDEA · NMS

NMS: una caja por objeto

Las cajas del detector



trazo continuo = se queda · discontinuo = se descarta

Orden por confianza: **A** 0,92 · **D** 0,88 · **B** 0,85 · **C** 0,60 · **E** 0,55

Paso	Se queda	Se descarta (IoU > 0,5)
1	A (0,92)	B (0,677) · C (0,620)
2	D (0,88)	E (0,565)
Resultado	A y D	una caja por sombrilla

Las que no se tocan con la elegida (IoU 0) pasan al paso siguiente.

Ojo con el umbral: dos sombrillas vecinas con IoU 0,25 se perderían con un umbral de 0,2.

LA IDEA · PRECISIÓN, RECALL Y AP

Medir un detector con números

Ejemplo: **5 sombrillas reales** y 6 cajas ordenadas por confianza (acierto = IoU ≥ 0,5). Valores acumulados hasta cada puesto.

Puesto	Confianza	¿Acierto?	Aciertos	Precisión	Recall
1	0,97	sí	1	1,00	0,20
2	0,91	no	1	0,50	0,20
3	0,84	sí	2	0,67	0,40
4	0,78	sí	3	0,75	0,60
5	0,66	no	3	0,60	0,60
6	0,52	sí	4	0,67	0,80

0,667

precisión final: 4 de 6, de lo que dice, qué parte es cierta

0,80

recall final: 4 de 5, de lo que hay, cuánto encuentra

0,633

AP: área bajo la curva precisión–recall

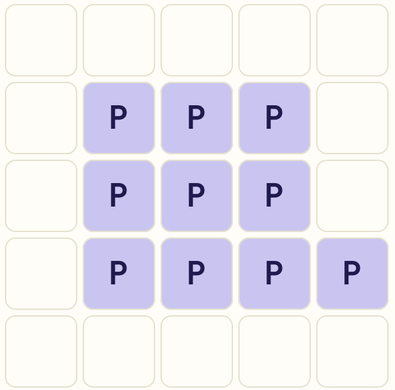
AP = 0,2 × (1 + 0,75 + 0,75 + 2/3): se suma 0,2 cada vez que sube el recall, con la mayor precisión que se consigue desde ahí.

LA IDEA · SEGMENTACIÓN

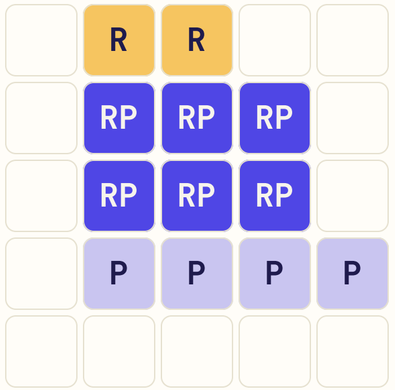
Segmentar: medir máscaras con IoU y Dice



Real: 8 píxeles



Predicha: 10 píxeles



Solape RP: 6 · unión: 12

IoU de máscaras

Píxeles de la **intersección** entre píxeles de la **unión**.

Dice

Dos veces la intersección entre la **suma** de las dos áreas. Muy usado en medicina.

IoU = 6 / 12 = **0,50** · **Dice** = 2·6 / (8 + 10) = **0,667**

El Dice da más peso a la intersección: vale igual o más que la IoU.

PRÁCTICA · CUADERNO S11_03

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar las herramientas	Clase
Ejercicio 1	La IoU de dos cajas	Clase
Ejercicio 2	NMS: una caja por objeto	Clase
Ejercicio 3	El AP (average precision)	Casa
Para leer	Detectores con OpenCV y su evaluación	Casa
Ejercicio 4	El mAP según el umbral de IoU	Casa
Ejercicio 5	IoU y Dice de máscaras	Casa
Para leer	Segmentación y YOLO (Colab)	Casa
Mini-reto	Contador de sombrillas	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**, cada ejercicio con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	La IoU de dos cajas	10 y 11
2	NMS	12 y 13
3	El AP	14 y 15
Ejemplo	Detectores con OpenCV	16
4	El mAP según la IoU	17 y 18
5	IoU y Dice de máscaras	19 y 20
Ejemplo	Segmentación y YOLO	21 y 22
Mini-reto	Contador de sombrillas	23 y 24

CONSULTA · EJERCICIO 1

La IoU de dos cajas (ejercicio 1)

```
def iou(a, b):
    ancho = max(0, min(a[2], b[2]) - max(a[0], b[0]))
    alto = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ancho * alto
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    union = area_a + area_b - inter
    return inter / union if union > 0 else 0.0
iou([0, 0, 10, 10], [5, 5, 15, 15])  # 0.143
```

25 · 175

intersección · unión

0,143

IoU

Una detección es correcta si su IoU con una caja real es ≥ 0,5.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la IoU (ejercicio 1)

max(0, min(...) - max(...))

Ancho y alto de la **intersección**. Con **max(0, ...)**, si las cajas no se tocan vale 0 y no un número negativo.

union = area_a + area_b - inter

Se **resta** la intersección: si no, se contaría dos veces.

Resultado de la prueba

[0,0,10,10] y **[5,5,15,15]**: intersección 5·5 = 25, unión 100 + 100 − 25 = 175 → **0,143**.

Convención

Una detección es correcta (verdadero positivo) si su IoU con una caja real es **≥ 0,5**.

CONSULTA · EJERCICIO 2

NMS: una caja por objeto (ejercicio 2)

```
def nms(cajas, puntuaciones, umbral_iou=0.5):
    orden = list(np.argsort(puntuaciones)[::-1])
    conservadas = []
    while orden:
        mejor = orden.pop(0)
        conservadas.append(int(mejor))
        orden = [i for i in orden if iou(cajas[mejor], cajas[i]) <= umbral_iou]
    return conservadas
_cajas = [[10, 10, 50, 50], [12, 12, 52, 52], [100, 100, 140, 140], [11, 9, 49, 51]]; _punt = [0.9, 0.8, 0.7, 0.6]
nms(_cajas, _punt)                    # [0, 2]
nms(_cajas, _punt, umbral_iou=0.99)  # [0, 1, 2, 3]
```

[0, 2]
umbral 0,5

[0, 1, 2, 3]
umbral 0,99

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer el NMS (ejercicio 2)

np.argsort(...)[::-1]

Ordena los índices de **mayor a menor** confianza. Sin ordenar no se sabría cuál es la mejor.

<= umbral_iou

Se quedan las cajas que apenas se solapan con la elegida; las que superan el umbral son duplicados y se descartan.

Con umbral 0,5

Se conservan **[0, 2]**: la 1 y la 3 eran duplicados de la 0, y la 2 está lejos.

Con umbral 0,99

Casi nada se considera duplicado: se conservan las **4** cajas.

CONSULTA · EJERCICIO 3

El AP, average precision (ejercicio 3)

```python
def average_precision(es_tp, n_reales):
    es_tp = np.asarray(es_tp, dtype=float)
    if len(es_tp) == 0 or n_reales == 0:
        return 0.0
    tp = np.cumsum(es_tp); fp = np.cumsum(1 - es_tp)
    precision = tp / (tp + fp); recall = tp / n_reales
    precision = np.maximum.accumulate(precision[::-1])[::-1]
    recall_previo = np.concatenate([[0.0], recall[:-1]])
    return float(np.sum((recall - recall_previo) * precision))
average_precision([1, 1, 0, 1, 0], 4)  # 0.6875
```

0,6875
AP de la tabla del enunciado

1 · 0,5
detector perfecto · solo la mitad

AP = 0,25·1 + 0,25·1 + 0,25·0,75: el falso positivo **no aumenta el recall.**

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer el AP (ejercicio 3)

maximum.accumulate(precision[::-1])[::-1]

Suaviza la precisión de derecha a izquierda: en cada punto vale la mejor precisión que se consigue **a partir de ahí**.

recall - recall_previo

El AP suma **aumento de recall × precisión**: un falso positivo no aumenta el recall, así que no añade nada.

Con [1, 1, 0, 1, 0] y 4 reales

AP = 0,25·1,00 + 0,25·1,00 + 0,25·0,75 = **0,6875**. La 5.ª detección falla y no suma.

Si falla

Solo encuentra la mitad de los objetos: AP = 0.5 → divides por las detecciones y no por **n_reales**.

CONSULTA · SECCIONES 5 Y 6 · DETECTORES CON OPENCV

Un detector con OpenCV, sin entrenar

Plantilla + NMS

Se desliza un círculo de color vivo por la imagen. En la escena de ejemplo: **47 cajas brutas** → **6** tras el NMS (4 sombrillas reales).

Color + contornos

HSV → máscara de color vivo → apertura → contornos → una caja por contorno.

Detector de color · 30 escenas	Valor
Sombrillas reales	171
Cajas detectadas	106
Precisión	0,887
Recall	0,550
AP	0,537

Acierta 94 de las 106 cajas, pero solo encuentra 94 de las 171 sombrillas: se le escapan las pastel y las solapadas.

CONSULTA · EJERCICIO 4

El mAP según el umbral de IoU (ejercicio 4)

```
umbrales = [0.5, 0.6, 0.7, 0.8, 0.9]
ap_por_umbral = [evaluar_detector(detector_color, umbral_iou=u)["AP"] for u in umbrales]
map_iou = float(np.mean(ap_por_umbral))    # 0.36
```

0,36

mAP (media de los 5 AP)

Umbral de IoU	0,5	0,6	0,7	0,8	0,9
AP del detector de color	0,54	0,51	0,48	0,29	0,00

El detector **encuentra** las sombrillas, pero sus cajas no son precisas.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer el mAP por umbral (ejercicio 4)

Un AP por umbral

umbral_iou=u: una caja solo acierta si su IoU llega a **u**. Cada vuelta es una evaluación más **exigente**.

np.mean(ap_por_umbral)

El mAP de este ejercicio es la **media de los 5 AP** (umbrales de 0,5 a 0,9).

Resultado

AP ≈ **0,54 · 0,51 · 0,48 · 0,29 · 0,00**; media **≈ 0,36**.

Por qué baja

Se exige que las cajas coincidan mucho más con las reales, y las del detector son **aproximadas**: con IoU ≥ 0,9 el AP cae a casi 0.

CONSULTA · EJERCICIO 5

IoU y Dice de máscaras (ejercicio 5)

```
def iou_mascara(pred, real):
    inter = np.logical_and(pred, real).sum()
    union = np.logical_or(pred, real).sum()
    return float(inter / union) if union > 0 else 1.0
def dice(pred, real):
    inter = np.logical_and(pred, real).sum()
    total = pred.sum() + real.sum()
    return float(2 * inter / total) if total > 0 else 1.0
iou_mascara(_p, _r)  # 0.333
dice(_p, _r)         # 0.5
```

1/3

IoU de las máscaras de prueba

0,5

Dice

El Dice vale igual o más que la IoU: da más peso a la intersección.

CONSULTA · ACLARACIONES · EJERCICIO 5

Leer IoU y Dice (ejercicio 5)

np.logical_and(pred, real).sum()

Cuenta los píxeles de la **intersección** (los dos a la vez). Con **logical_or**, los de la **unión**.

if union > 0 else 1.0

Dos máscaras **vacías** coinciden: devuelve 1.0 y evita dividir entre 0.

Máscaras de prueba

Intersección 1, unión 3 → IoU **1/3**. Dice = 2·1 / (2 + 2) = **0,5**.

Si falla

2·1 / (2 + 2) = 0.5 → falta el factor 2 o divides entre la unión.

CONSULTA · SECCIÓN 6 · SEGMENTACIÓN

Segmentar con color y con una mini red

0,629

IoU media del umbral de color (Dice medio: 0,756)

0,612

IoU de la mini red en 60 escenas nuevas

0,587

IoU del umbral de color en esas mismas 60

Sin entrenar nada

La máscara del detector de color, más una limpieza, ya es una **segmentación**. La mini red **encoder-decoder** reduce la imagen con convoluciones y la agranda hasta dar una probabilidad por píxel.

Casi iguales: en un problema limpio, un método clásico bien pensado es un buen punto de partida.

CONSULTA · SECCIÓN 7 · YOLO (COLAB)

YOLO: el detector moderno

	Detector clásico (OpenCV)	Detector moderno (YOLO)
Qué aprende	Nada: reglas hechas a mano (color, plantilla)	Todo el detector, con miles de imágenes etiquetadas con cajas
Cajas por imagen	Las de la regla; el NMS lo aplicas tú	Muchas en una pasada; ya aplica NMS por dentro
Va mejor cuando	El objeto tiene un color inconfundible	Hay sombras, oclusiones o fondos complejos
Se evalúa con	AP con IoU ≥ 0,5	mAP@[.5:.95]

La celda de YOLO necesita internet y no está verificada en el taller: se prueba en Colab con USAR_INTERNET = True.

Lo que programaste a mano (**IoU, NMS y AP**) es justo lo que ocurre dentro y alrededor de YOLO.

CONSULTA · MINI-RETO

Contador de sombrillas (mini-reto)

```
f1_por_sat = {}
for s in (60, 90, 120, 150, 180):
    r = evaluar_detector(lambda im, s=s: detectar_color(im, sat_min=s)[:2], n_escenas=30, semilla=500)
    p, rec = r["precisión"], r["recall"]
    f1_por_sat[s] = 2 * p * rec / (p + rec) if p + rec > 0 else 0.0
mejor_sat = max(f1_por_sat, key=f1_por_sat.get)  # 90
{s: round(v, 2) for s, v in f1_por_sat.items()}  # {60: 0.08, 90: 0.71, 120: 0.69, 150: 0.69, 180: 0.69}
```

90
mejor **sat_min**

0,71
F1 con ese valor

1,87 por foto
error medio de conteo

Con sat_min = 60 la arena «cuela» como sombrilla: precisión 0,07 y F1 0,08.

CONSULTA · ACLARACIONES · MINI-RETO

Leer el mini-reto

Error de conteo

Por escena, **cajas detectadas − sombrillas reales** en valor absoluto; se promedia: **1,87** sombrillas por foto.

f1_por_sat

Para cada **sat_min**, F1 = 2·p·r / (p + r). El mejor es **90**, con F1 **0,71**.

sat_min = 60

La arena **cuela** como sombrilla: 225 cajas para 165 reales (precisión 0,07) y el F1 se hunde a **0,08**.

De 120 a 180

El F1 no cambia (**0,69**): el detector ya solo ve colores muy saturados y **no encuentra más** sombrillas (recall 0,58).